PYTHON ACTIVITY-BASED LEARNING :

CASE STUDY ASSIGNMENT :

Subject: Python Programming

Submitted By: Shaik Irshad

Roll Number: 26001618

Branch: AIML

Submitted To: ASWINI mam

Academic Year: 2026–2027

LIST OF SELECTED CASE STUDIES :

1. Employee Payroll and Deduction Analysis
2. ATM Transaction Sequence Simulation
3. Repeated Transaction Detection
4. Multi-Stage Examination Eligibility System
5. Tuple-Based Product Catalogue Audit
6. Food Waste Monitoring System
7. Student Academic Risk Classification
8. Smart Electricity Billing for Multiple Apartments
9. Environmental Monitoring Data Analyzer
10. Advanced String-Based Registration Record Analyzer

CASE STUDY 1: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement :

To develop a Python program to calculate an employee's gross salary based on basic pay, allowances, overtime pay, and incentives. The program should apply deductions according to the gross salary, calculate the net salary, and display a properly formatted salary statement.

2. Objectives :

1. To calculate the gross salary of an employee.

2. To apply deductions based on the gross salary.

3. To calculate the employee's net salary.

4. To display salary details in a clear and organized format.

5. To understand the use of Python variables, data types, arithmetic operators, and conditional statements.

3. Assumptions :

1. The employee's name and salary components are entered by the user.

2. Basic pay, allowances, overtime pay, and incentives are non-negative values.

3. Gross salary is calculated by adding all salary components.

4. If the gross salary is greater than or equal to ₹50,000, a deduction rate of 10% is applied.

5. If the gross salary is below ₹50,000, a deduction rate of 5% is applied.

6. Net salary is calculated by subtracting the deduction amount from the gross salary.

7. The deduction rates are assumed for this academic case study and do not represent actual payroll or tax rules.

4. Algorithm :

Step 1: Start the program.

Step 2: Read the employee's name.

Step 3: Input basic pay, allowances, overtime pay, and incentives.

Step 4: Calculate the gross salary by adding all salary components.

Step 5: Check whether the gross salary is greater than or equal to ₹50,000.

Step 6: If the condition is true, calculate the deduction at 10%. Otherwise, calculate the deduction at 5%.

Step 7: Calculate the net salary by subtracting the deduction from the gross salary.

Step 8: Display the employee's salary statement, including gross salary, deduction, and net salary.

Step 9: Stop the program.

5. Pseudocode :

```
BEGIN
INPUT employee_name
INPUT basic_pay
INPUT allowances
INPUT overtime_pay
INPUT incentives
gross_salary = basic_pay + allowances
             + overtime_pay + incentives
IF gross_salary >= 50000 THEN
   deduction_rate = 0.10
ELSE
   deduction_rate = 0.05
END IF
deduction = gross_salary * deduction_rate
net_salary = gross_salary - deduction
DISPLAY employee_name
DISPLAY gross_salary
DISPLAY deduction
DISPLAY net_salary
END
```

6. Flowchart :

Start

↓

Input Employee Name

↓

Input Basic Pay, Allowances, Overtime Pay and Incentives

↓

Calculate Gross Salary

↓

Is Gross Salary ≥ ₹50,000?

↓

Yes: Deduction Rate = 10%

No: Deduction Rate = 5%

↓

Calculate Deduction

↓

Calculate Net Salary

↓

Display Salary Statement

↓

Stop

7. Python Source Code :

```
# Employee Payroll and Deduction Analysis
employee_name = input("Enter employee name: ")
basic_pay = float(input("Enter basic pay: "))
allowances = float(input("Enter allowances: "))
overtime_pay = float(input("Enter overtime pay: "))
incentives = float(input("Enter incentives:"))
gross_salary = (
    basic_pay + allowances + overtime_pay + incentives
```

```
)
if gross_salary >= 50000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.05
deduction = gross_salary * deduction_rate
net_salary = gross_salary - deduction
print("Employee Name :", employee_name)
print("Basic Pay     : Rs.", basic_pay)
print("Allowances    : Rs.", allowances)
print("Overtime Pay  : Rs.", overtime_pay)
print("Incentives    : Rs.", incentives)
print("Gross Salary  : Rs.", gross_salary)
print("Deduction     : Rs.", deduction)
print("Net Salary    : Rs.", net_salary)
```

8. Test Cases :

Test Case 1 :

- Employee Name: Ravi
- Basic Pay: ₹25,000
- Allowances: ₹5,000
- Overtime Pay: ₹2,000
- Incentives: ₹3,000
- Expected Gross Salary: ₹35,000
- Deduction Rate: 5%
- Deduction Amount: ₹1,750
- Net Salary: ₹33,250

Test Case 2 :

- Employee Name: Ravi
- Basic Pay: ₹30,000

- Allowances: ₹10,000

- Overtime Pay: ₹5,000

- Incentives: ₹5,000

- Expected Gross Salary: ₹50,000

- Deduction Rate: 10%

- Deduction Amount: ₹5,000

- Net Salary: ₹45,000

Test Case 3 :

- Employee Name: Ravi

- Basic Pay: ₹40,000

- Allowances: ₹10,000

- Overtime Pay: ₹5,000

- Incentives: ₹5,000

- Expected Gross Salary: ₹60,000

- Deduction Rate: 10%

- Deduction Amount: ₹6,000

- Net Salary: ₹54,000

9. Output :

```
Enter employee name: irshad
Enter basic pay: 30000
Enter allowances: 10000
Enter overtime pay: 5000
Enter incentives: 5000
Employee Name : irshad
Basic Pay     : Rs. 30000.0
Allowances    : Rs. 10000.0
Overtime Pay  : Rs. 5000.0
Incentives    : Rs. 5000.0
Gross Salary  : Rs. 50000.0
Deduction     : Rs. 5000.0
Net Salary    : Rs. 45000.0
```

10. Conclusion :

The Employee Payroll and Deduction Analysis program successfully calculates the gross salary, applicable deductions, and net salary of an employee. It displays salary details in an organized format. This case study demonstrates the practical

application of Python variables, data types, arithmetic operators, user input, and conditional statements in solving a real-world payroll problem.

CASE STUDY 2: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement :

To develop a Python program that simulates ATM transactions by processing a sequence of deposits and withdrawals. The program should maintain the account balance, enforce withdrawal limits, reject invalid transactions, and display a summary of all transactions performed.

2. Objectives :

1. To simulate ATM deposit and withdrawal transactions.

2. To maintain and update the account balance after every transaction.

3. To enforce the specified withdrawal limit.

4. To reject invalid transactions and withdrawals exceeding the available balance.

5. To display the final account balance and transaction summary.

6. To understand the use of Python variables, operators, loops, lists, and conditional statements.

3. Assumptions :

1. The initial account balance is ₹10,000.

2. The program processes five transactions.

3. Each transaction is either a deposit or a withdrawal.

4. The transaction amount must be greater than zero.

5. The maximum amount allowed for a single withdrawal is ₹5,000.

6. A withdrawal is allowed only when the amount does not exceed the available balance.

7. Invalid transactions are rejected without changing the account balance.

8. All amounts are entered in Indian Rupees (₹).

4. Algorithm :

Step 1: Start the program.

Step 2: Initialize the account balance to ₹10,000.

Step 3: Set the maximum withdrawal limit to ₹5,000.

Step 4: Repeat the transaction process five times.

Step 5: Read the transaction type and amount.

Step 6: Check whether the transaction amount is greater than zero.

Step 7: If the transaction is a deposit, add the amount to the account balance.

Step 8: If the transaction is a withdrawal, check the available balance and withdrawal limit.

Step 9: If the withdrawal is valid, subtract the amount from the balance. Otherwise, reject the transaction.

Step 10: Store the transaction details.

Step 11: Display the final account balance and transaction summary.

Step 12: Stop the program.

5. Pseudocode :

```
BEGIN
    balance = 10000
    withdrawal_limit = 5000
    transactions = EMPTY LIST
        REPEAT 5 TIMES
        INPUT transaction_type
        INPUT amount
        IF amount <= 0 THEN
            DISPLAY "Invalid amount"
        ELSE IF transaction_type = "deposit" THEN
            balance = balance + amount
            STORE transaction details
            DISPLAY "Deposit successful"
        ELSE IF transaction_type = "withdrawal" THEN
```

```
            IF amount > withdrawal_limit THEN
                DISPLAY "Withdrawal limit exceeded"
            ELSE IF amount > balance THEN
                DISPLAY "Insufficient balance"
            ELSE
                balance = balance - amount
                STORE transaction details
                DISPLAY "Withdrawal successful"
            END IF
        ELSE
            DISPLAY "Invalid transaction type"
        END IF
    END REPEAT
    DISPLAY transactions
    DISPLAY balance
END
```

6. Flowchart :

Start

↓

Initialize Balance = ₹10,000

↓

Set Withdrawal Limit = ₹5,000

↓

Enter Transaction Type and Amount

↓

Is Amount Greater Than Zero?

- No → Display "Invalid Amount"

- Yes → Check Transaction Type

↓

Is It a Deposit?

- Yes → Add Amount to Balance

- No → Check Whether It Is a Withdrawal

↓

For Withdrawal, Check Balance and Withdrawal Limit

- Valid → Subtract Amount from Balance

- Invalid → Display Rejection Message

↓

Store Valid Transaction Details

↓

Are Five Transactions Completed?

- No → Enter Next Transaction

- Yes → Display Transaction Summary and Final Balance

↓

Stop

7. Python Source Code :

```
# ATM Transaction Sequence Simulation
balance = 10000
withdrawal_limit = 5000
transactions = []
for i in range(5):
    print("Transaction", i + 1)
    transaction_type = input("Enter deposit or withdrawal: ").lower()
    amount = float(input("Enter amount: "))
    if amount <= 0:
        print("Invalid amount")
        transactions.append("Invalid amount”)
    elif transaction_type == "deposit":
        balance += amount
```

```
        print("Deposit successful")
        transactions.append("Deposit", amount, balance)
    elif transaction_type == "withdrawal":
        if amount > withdrawal_limit:
            print("Withdrawal limit exceeded")
            transactions.append("Withdrawal limit exceeded”)
        elif amount > balance:
            print("Insufficient balance")
            transactions.append("Insufficient balance")
        else:
            balance -= amount
            print("Withdrawal successful")
            transactions.append("Withdrawal", amount, balance)
else:
        print("Invalid transaction type")
        transactions.append("Invalid transaction type”)
print("Transaction Summary")
for transaction in transactions:
    print(transaction)
print("Final Account Balance: Rs.", balance)
```

8. Test Cases :

Test Case 1:

Deposit

Initial Balance = ₹10,000

Transaction Type = Deposit

Amount = ₹2,000

Expected Balance = ₹12,000

Result = Deposit successful

Test Case 2:

Valid Withdrawal

Initial Balance = ₹10,000

Transaction Type = Withdrawal

Amount = ₹3,000

Expected Balance = ₹7,000

Result = Withdrawal successful

Test Case 3:

Withdrawal Limit Exceeded

Initial Balance = ₹10,000

Transaction Type = Withdrawal

Amount = ₹6,000

Expected Balance = ₹10,000

Result = Withdrawal limit exceeded

Test Case 4:

Insufficient Balance

Initial Balance = ₹4,000

Transaction Type = Withdrawal

Amount = ₹4,500

Expected Balance = ₹4,000

Result = Insufficient balance

Test Case 5:

Invalid Amount

Initial Balance = ₹10,000

Transaction Type = Deposit

Amount = ₹0

Expected Balance = ₹10,000

Result = Invalid amount

9. Output :

```
Transaction 1
Enter deposit or withdrawal: deposit
Enter amount: 2000
Deposit successful

Transaction 2
Enter deposit or withdrawal: withdrawal
Enter amount: 3000
Withdrawal successful

Transaction 3
Enter deposit or withdrawal: withdrawal
Enter amount: 6000
Withdrawal limit exceeded

Transaction 4
Enter deposit or withdrawal: deposit
Enter amount: 1000
Deposit successful

Transaction 5
Enter deposit or withdrawal: withdrawal
Enter amount: 500
Withdrawal successful

Transaction Summary
('Deposit', 2000.0, 12000.0)
('Withdrawal', 3000.0, 9000.0)
Withdrawal limit exceeded
('Deposit', 1000.0, 10000.0)
('Withdrawal', 500.0, 9500.0)
Final Account Balance: Rs. 9500.0
```

10. Conclusion :

The ATM Transaction Sequence Simulation program successfully processes deposits and withdrawals while maintaining the account balance. It checks the withdrawal limit, rejects invalid transactions, and displays a transaction summary. This case study demonstrates the use of Python variables, operators, loops, lists, and conditional statements in simulating ATM operations.

CASE STUDY 3: REPEATED TRANSACTION DETECTION :

1. Problem Statement :

To develop a Python program that analyses a sequence of financial transactions, identifies repeated transactions using transaction details, detects invalid transaction amounts, and displays a summary of unique and repeated transactions.

2. Objectives :

1. To store transaction details using Python lists.

2. To identify repeated transactions in a sequence.

3. To detect transactions with invalid amounts.

4. To count the number of unique and repeated transactions.

5. To display a clear transaction analysis report.

6. To understand the use of Python variables, lists, loops, conditional statements, and operators.

3. Assumptions :

1. The program processes a predefined list of transactions.

2. Each transaction contains a transaction ID and an amount.

3. A transaction is considered repeated if the same transaction ID appears more than once.

4. Transaction amounts must be greater than zero to be valid.

5. Invalid transactions are reported separately.

6. The program displays the total number of unique, repeated, and invalid transactions.

4. Algorithm :

Step 1: Start the program.

Step 2: Initialize a list containing transaction IDs and their amounts.

Step 3: Create an empty list to store the transaction IDs already processed.

Step 4: Initialize counters for repeated and invalid transactions.

Step 5: Read each transaction from the transaction list.

Step 6: Check whether the transaction amount is greater than zero.

Step 7: If the amount is invalid, display an invalid transaction message and increase the invalid counter.

Step 8: If the transaction ID has already been processed, identify it as a repeated transaction and increase the repeated counter.

Step 9: Otherwise, store the transaction ID as a processed transaction.

Step 10: Display the transaction details and the final summary.

Step 11: Stop the program.

5. Pseudocode :

```
BEGIN
    transactions = [
        ("T101", 500),
        ("T102", 1000),
        ("T101", 500),
        ("T103", -200),
        ("T104", 750),
        ("T102", 1000)
    ]
    processed_ids = EMPTY LIST
    repeated_count = 0
    invalid_count = 0
    FOR each transaction IN transactions
        READ transaction_id, amount
        IF amount <= 0 THEN
            DISPLAY "Invalid transaction"
            invalid_count = invalid_count + 1
        ELSE IF transaction_id IN processed_ids THEN
            DISPLAY "Repeated transaction"
             repeated_count = repeated_count + 1
        ELSE
            ADD transaction_id TO processed_ids
            DISPLAY "Valid transaction"
        END IF
    END FOR
    DISPLAY processed_ids
    DISPLAY repeated_count
```

DISPLAY invalid_count

END

6. Flowchart :

Start

↓

Initialize Transaction List

↓

Initialize Processed IDs and Counters

↓

Read Next Transaction

↓

Is Amount ≤ 0?

Yes → Display Invalid Transaction → Increase Invalid Counter

No → Check Whether Transaction ID Already Exists

↓

Is Transaction ID Already Processed?

Yes → Display Repeated Transaction → Increase Repeated Counter

No → Add ID to Processed List → Display Valid Transaction

↓

Are All Transactions Processed?

No → Read Next Transaction

Yes → Display Transaction Summary

↓

Stop

7. Python Source Code :

```
# Repeated Transaction Detection
transactions = [
    ("T101", 500),
    ("T102", 1000),
```

```
    ("T101", 500),
    ("T103", -200),
    ("T104", 750),
    ("T102", 1000)
]
processed_ids = []
repeated_count = 0
invalid_count = 0
for transaction in transactions:
    transaction_id, amount = transaction
    if amount <= 0:
        print(transaction_id, "- Invalid transaction")
        invalid_count += 1
    elif transaction_id in processed_ids:
        print(transaction_id, "- Repeated transaction")
        repeated_count += 1
    else:
        processed_ids.append(transaction_id)
        print(transaction_id, "- Valid transaction")
print("Transaction Summary")
print("Unique transactions:", len(processed_ids))
print("Repeated transactions:", repeated_count)
print("Invalid transactions:", invalid_count)
```

8. Test Cases :

Test Case 1:

Valid Transaction

Transaction ID = T101

Amount = ₹500

Expected Result = Valid transaction

Test Case 2:

Repeated Transaction

Transaction ID = T101

Amount = ₹500

Expected Result = Repeated transaction

Test Case 3:

Invalid Transaction

Transaction ID = T103

Amount = -₹200

Expected Result = Invalid transaction

Test Case 4:

New Transaction

Transaction ID = T104

Amount = ₹750

Expected Result = Valid transaction

9. Output :

```
T101 - Valid transaction
T102 - Valid transaction
T101 - Repeated transaction
T103 - Invalid transaction
T104 - Valid transaction
T102 - Repeated transaction

Transaction Summary
Unique transactions: 3
Repeated transactions: 2
Invalid transactions: 1
```

10. Conclusion :

The Repeated Transaction Detection program successfully identifies repeated transaction IDs and invalid transaction amounts. It maintains a record of processed transactions and displays a summary of unique, repeated, and invalid transactions. This case study demonstrates the use of Python lists, loops, conditional statements, and tuple unpacking for transaction analysis.

CASE STUDY 4: MULTI-STAGE EXAMINATION ELIGIBILITY SYSTEM

1. Problem Statement :

To develop a Python program that determines students' eligibility to attend examinations by checking their attendance percentage, internal marks, and assignment completion status. The program should apply the specified eligibility conditions, classify students as eligible or not eligible, and display the reasons for ineligibility.

2. Objectives :

1. To determine examination eligibility based on multiple conditions.

2. To check whether students meet the minimum attendance requirement.

3. To verify whether students meet the minimum internal marks requirement.

4. To check assignment completion status.

5. To classify students as eligible or not eligible.

6. To display the reasons for ineligibility.

7. To understand the use of variables, operators, conditional statements, and logical operators in Python.

3. Assumptions :

1. Student details are predefined in the program.

2. The minimum attendance requirement is 75%.

3. The minimum internal marks required are 20 out of 40.

4. Students must complete their assignments to be eligible.

5. A student must satisfy all three conditions to attend the examination.

6. If any condition is not satisfied, the student is classified as not eligible.

7. The program displays the student's eligibility status and the reasons for ineligibility.

4. Algorithm :

Step 1: Start.

Step 2: Initialize the student details, including name, attendance percentage, internal marks, and assignment status.

Step 3: Set the minimum attendance requirement to 75%.

Step 4: Set the minimum internal marks requirement to 20.

Step 5: Check whether the student's attendance is at least 75%.

Step 6: Check whether the student's internal marks are at least 20.

Step 7: Check whether the student has completed all assignments.

Step 8: If all three conditions are satisfied, display "Eligible for Examination".

Step 9: Otherwise, display "Not Eligible for Examination" and the reasons for ineligibility.

Step 10: Repeat the process for all students.

Step 11: Stop.

5. Pseudocode :

```
BEGIN
students = [
    ("Anu", 85, 30, True),
    ("Ravi", 65, 25, True),
    ("Sita", 80, 15, True),
    ("Kiran", 90, 32, False)
]
FOR each student IN students
    READ name, attendance, marks, assignment_status
    IF attendance >= 75 AND marks >= 20 AND assignment_status == True THEN
        DISPLAY name
        DISPLAY "Eligible for Examination"
    ELSE
        DISPLAY name
        DISPLAY "Not Eligible for Examination"
        IF attendance < 75 THEN
            DISPLAY "Attendance is below 75%"
        END IF
        IF marks < 20 THEN
```

```
            DISPLAY "Internal marks are below 20"
        END IF
        IF assignment_status == False THEN
            DISPLAY "Assignments are incomplete"
        END IF
    END IF
END FOR
END
```

6. Flowchart :

Start

↓

Initialize Student Details

↓

Set Minimum Attendance = 75% and Minimum Marks = 20

↓

Read Student Details

↓

Check Attendance, Internal Marks, and Assignment Status

↓

Are All Three Conditions Satisfied?

- Yes: Display "Eligible for Examination".

- No: Display "Not Eligible for Examination" and the reasons.

↓

Are All Students Processed?

- No: Read the next student's details.

- Yes: Stop.

7. Python Source Code :

```
# Multi-Stage Examination Eligibility System
students = [
```

```
    ("Anu", 85, 30, True),
    ("Ravi", 65, 25, True),
    ("Sita", 80, 15, True),
    ("Kiran", 90, 32, False)
]
minimum_attendance = 75
minimum_marks = 20
for student in students:
    name, attendance, marks, assignment_status = student
    print("\nStudent Name:", name)
    if (attendance >= minimum_attendance
            and marks >= minimum_marks
            and assignment_status == True):
        print("Eligible for Examination")
    else:
        print("Not Eligible for Examination")
        if attendance < minimum_attendance:
            print("Reason: Attendance is below 75%")
        if marks < minimum_marks:
            print("Reason: Internal marks are below 20")
        if assignment_status == False:
            print("Reason: Assignments are incomplete")
```

8. Test Cases :

Test Case	Attendance	Internal Marks	Assignments Completed	Expected Result
1	85%	30	Yes	Eligible
2	65%	25	Yes	Not Eligible — Low Attendance
3	80%	15	Yes	Not Eligible — Low Marks

4 | 90% | 32 | No | Not Eligible — Incomplete Assignments

5 | 75% | 20 | Yes | Eligible

9. Output :

```
Student Name: Anu
Eligible for Examination

Student Name: Ravi
Not Eligible for Examination
Reason: Attendance is below 75%

Student Name: Sita
Not Eligible for Examination
Reason: Internal marks are below 20

Student Name: Kiran
Not Eligible for Examination
Reason: Assignments are incomplete
```

10. Conclusion :

The Multi-Stage Examination Eligibility System was successfully developed using Python. The program checks attendance, internal marks, and assignment completion to determine examination eligibility. It displays the eligibility status and identifies the reasons when a student is not eligible. This program demonstrates the practical use of variables, tuples, loops, conditional statements, and logical operators in Python.

CASE STUDY 5: TUPLE-BASED PRODUCT CATALOGUE AUDIT

1. Problem Statement :

To develop a Python program that stores product identifiers, prices, and stock quantities using tuples. The program should identify invalid product records, calculate the total inventory value, and identify items requiring replenishment.

2. Objectives :

1. To store product details using Python tuples.

2. To identify invalid product records.

3. To calculate the total inventory value of all valid products.

4. To identify products with stock quantities below the minimum required level.

5. To display product details and inventory analysis in a clear format.

6. To understand the use of tuples, loops, conditional statements, and arithmetic operators in Python.

3. Assumptions :

1. Product details are stored in a predefined list of tuples.

2. Each tuple contains a product identifier, price, and stock quantity.

3. Product identifiers must not be empty.

4. Product prices must be greater than zero.

5. Stock quantities must not be negative.

6. The minimum stock quantity required is assumed to be 10 units.

7. Inventory value is calculated by multiplying the product price by its stock quantity.

8. Products with fewer than 10 units in stock are identified as requiring replenishment.

4. Algorithm :

Step 1 : Start.

Step 2 : Initialize a list of product tuples containing product ID, price, and stock quantity.

Step 3 : Initialize the total inventory value to zero.

Step 4 : Read each product record from the list.

Step 5 : Check whether the product ID is valid, the price is greater than zero, and the stock quantity is non-negative.

Step 6 : If the record is invalid, display an invalid product message.

Step 7 : Otherwise, calculate the product's inventory value by multiplying its price by its stock quantity.

Step 8 : Add the product's inventory value to the total inventory value.

Step 9 : Check whether the stock quantity is below 10 units. If it is, display that the product requires replenishment.

Step 10 : Repeat the process for all product records.

Step 11 : Display the total inventory value.

Step 12 : Stop.

5. Pseudocode :

```
BEGIN
    products = [
        ("P101", 100, 20),
        ("P102", 250, 5),
        ("P103", -50, 10),
        ("P104", 500, 0),
        ("P105", 150, 8)
    ]
    total_inventory_value = 0
    minimum_stock = 10
    FOR each product IN products
        READ product_id, price, quantity
        IF product_id is empty OR price <= 0 OR quantity < 0 THEN
            DISPLAY "Invalid product record"
        ELSE
            inventory_value = price * quantity
            total_inventory_value =
                total_inventory_value + inventory_value
            DISPLAY product details
            DISPLAY inventory_value
            IF quantity < minimum_stock THEN
                DISPLAY "Replenishment Required"
            END IF
        END IF
    END FOR
    DISPLAY total_inventory_value
END
```

6. Flowchart :

Start

↓

Initialize Product Tuple List

↓

Initialize Total Inventory Value = 0

↓

Read Product Record

↓

Check Product ID, Price, and Stock Quantity

↓

Is the Product Record Valid?

- No : Display Invalid Product Record.

- Yes : Calculate Inventory Value and update the total.

↓

Is Stock Quantity Less Than 10?

- Yes : Display Replenishment Required.

- No : Continue to the next product.

↓

Are All Product Records Processed?

- No : Read the next product.

- Yes : Display Total Inventory Value.

↓

Stop

7. Python Source Code :

```
# Tuple-Based Product Catalogue Audit
products = [
    ("P101", 100, 20),
    ("P102", 250, 5),
```

```
    ("P103", -50, 10),
    ("P104", 500, 0),
    ("P105", 150, 8)
]
total_inventory_value = 0
minimum_stock = 10
for product in products:
    product_id, price, quantity = product
    print("Product ID:", product_id)
    if product_id == "" or price <= 0 or quantity < 0:
        print("Invalid product record")
    else:
        inventory_value = price * quantity
        total_inventory_value += inventory_value
        print("Price: Rs.", price)
        print("Stock Quantity:", quantity)
        print("Inventory Value: Rs.", inventory_value)
        if quantity < minimum_stock:
            print("Replenishment Required")
print("Total Inventory Value: Rs.", total_inventory_value)
```

8. Test Cases :

Test Case	Product ID	Price	Stock Quantity	Expected Result
1.	P101	₹100	20	Valid record
2.	P102	₹250	5	Replenishment required
3.	P103	-₹50	10	Invalid product record
4.	P104	₹500	0	Valid record; replenishment required
5.	P105	₹150	8	Replenishment required

9. Output :

```
Product ID: P101
Price: Rs. 100
Stock Quantity: 20
Inventory Value: Rs. 2000
Product ID: P102
Price: Rs. 250
Stock Quantity: 5
Inventory Value: Rs. 1250
Replenishment Required
Product ID: P103
Invalid product record
Product ID: P104
Price: Rs. 500
Stock Quantity: 0
Inventory Value: Rs. 0
Replenishment Required
Product ID: P105
Price: Rs. 150
Stock Quantity: 8
Inventory Value: Rs. 1200
Replenishment Required
Total Inventory Value: Rs. 4450
```

10. Conclusion :

The Tuple-Based Product Catalogue Audit program successfully stores and analyses product details using tuples. It identifies invalid product records, calculates the total inventory value, and detects products requiring replenishment. This case study demonstrates the practical use of Python tuples, tuple unpacking, loops, conditional statements, and arithmetic operators in inventory management.

CASE STUDY 6: FOOD WASTE MONITORING SYSTEM

1. Problem Statement :

To develop a Python program that monitors food preparation, meals served, and leftover food over several days. The program should calculate the food wastage percentage for each day and identify the days on which food wastage exceeds the acceptable limit.

2. Objectives :

1. To store daily food preparation, meals served, and leftover quantities.

2. To calculate the food wastage percentage for each day.

3. To identify days with excessive food wastage.

4. To calculate the total food prepared, served, and wasted.

5. To display a summary of food waste monitoring.

6. To understand the use of Python lists, loops, conditional statements, and arithmetic operators.

3. Assumptions :

1. Food monitoring data for five days is stored in a predefined list.

2. Food quantities are measured in meals.

3. Leftover meals are calculated by subtracting meals served from meals prepared.

4. Food wastage percentage is calculated using the number of leftover meals divided by the number of meals prepared, multiplied by 100.

5. The acceptable food wastage limit is assumed to be 10%.

6. If the wastage percentage exceeds 10%, the day is classified as having excessive food wastage.

7. The number of meals prepared must be greater than zero, and meals served cannot exceed meals prepared.

4. Algorithm :

Step 1 : Start.

Step 2 : Initialize a list containing daily food preparation and meals served.

Step 3 : Initialize total prepared meals, total served meals, and total leftover meals to zero.

Step 4 : Set the acceptable wastage limit to 10%.

Step 5 : Read the food data for each day.

Step 6 : Calculate leftover meals by subtracting meals served from meals prepared.

Step 7 : Calculate the food wastage percentage.

Step 8 : Compare the wastage percentage with the acceptable limit.

Step 9 : If the wastage percentage exceeds 10%, display an excessive food wastage warning.

Step 10 : Update the total prepared, served, and leftover meals.

Step 11 : Display the daily results and overall summary.

Step 12 : Stop.

5. Pseudocode :

```
BEGIN
    food_data = [
        ("Day 1", 100, 90),
        ("Day 2", 120, 100),
        ("Day 3", 80, 76),
        ("Day 4", 150, 120),
        ("Day 5", 100, 95)
    ]
    wastage_limit = 10
    total_prepared = 0
    total_served = 0
    total_leftover = 0
    FOR each day IN food_data
        READ day_name, prepared, served

        leftover = prepared - served
        wastage_percentage = (leftover / prepared) * 100
        DISPLAY day_name
        DISPLAY prepared, served, leftover
        DISPLAY wastage_percentage
        IF wastage_percentage > wastage_limit THEN
            DISPLAY "Excessive Food Wastage"
        ELSE
            DISPLAY "Food Wastage Within Limit"
        END IF
        total_prepared = total_prepared + prepared
        total_served = total_served + served
        total_leftover = total_leftover + leftover
    END FOR
```

DISPLAY total_prepared

DISPLAY total_served

DISPLAY total_leftover

END

6. Flowchart :

Start

↓

Initialize Daily Food Data

↓

Set Wastage Limit = 10%

↓

Initialize Total Counters

↓

Read Daily Food Data

↓

Calculate Leftover Meals

↓

Calculate Wastage Percentage

↓

Is Wastage Percentage Greater Than 10%?

- Yes : Display "Excessive Food Wastage".

- No : Display "Food Wastage Within Limit".

↓

Update Total Prepared, Served, and Leftover Meals

↓

Are All Days Processed?

- No : Read the Next Day's Data.

- Yes : Display Overall Summary.

↓

Stop

7. Python Source Code :

```
# Food Waste Monitoring System
food_data = [
    ("Day 1", 100, 90),
    ("Day 2", 120, 100),
    ("Day 3", 80, 76),
    ("Day 4", 150, 120),
    ("Day 5", 100, 95)
]
wastage_limit = 10
total_prepared = 0
total_served = 0
total_leftover = 0
for day in food_data:
    day_name, prepared, served = day
    leftover = prepared - served
    wastage_percentage = (leftover / prepared) * 100
    print(day_name)
    print("Meals Prepared:", prepared)
    print("Meals Served:", served)
    print("Leftover Meals:", leftover)
    print("Wastage Percentage:", round(wastage_percentage, 2), "%")
    if wastage_percentage > wastage_limit:
        print("Excessive Food Wastage")
    else:
        print("Food Wastage Within Limit")
    total_prepared += prepared
    total_served += served
```

```
    total_leftover += leftover
print("Overall Food Waste Summary")
print("Total Meals Prepared:", total_prepared)
print("Total Meals Served:", total_served)
print("Total Leftover Meals:", total_leftover)
overall_wastage = (total_leftover / total_prepared) * 100
print("Overall Wastage Percentage:", round(overall_wastage, 2), "%")
```

8. Test Cases :

Test Case 1 :

Meals Prepared : 100

Meals Served : 90

Expected Result : Food Wastage Within Limit.

Test Case 2 :

Meals Prepared : 120

Meals Served : 100

Expected Result : Excessive Food Wastage.

Test Case 3 :

Meals Prepared : 80

Meals Served : 76

Expected Result : Food Wastage Within Limit.

Test Case 4 :

Meals Prepared : 150

Meals Served : 120

Expected Result : Excessive Food Wastage.

Test Case 5 :

Meals Prepared : 100

Meals Served : 95

Expected Result : Food Wastage Within Limit.

9. Output :

```
Day 1
Meals Prepared: 100
Meals Served: 90
Leftover Meals: 10
Wastage Percentage: 10.0 %
Food Wastage Within Limit
Day 2
Meals Prepared: 120
Meals Served: 100
Leftover Meals: 20
Wastage Percentage: 16.67 %
Excessive Food Wastage
Day 3
Meals Prepared: 80
Meals Served: 76
Leftover Meals: 4
Wastage Percentage: 5.0 %
Food Wastage Within Limit
Day 4
Meals Prepared: 150
Meals Served: 120
Leftover Meals: 30
Wastage Percentage: 20.0 %
Excessive Food Wastage
Day 5
Meals Prepared: 100
Meals Served: 95
Leftover Meals: 5
Wastage Percentage: 5.0 %
Food Wastage Within Limit
Overall Food Waste Summary
Total Meals Prepared: 550
Total Meals Served: 481
Total Leftover Meals: 69
Overall Wastage Percentage: 12.55 %
```

10. Conclusion :

The Food Waste Monitoring System successfully calculates daily leftover meals and food wastage percentages. It identifies days when wastage exceeds the acceptable limit and displays an overall summary. This case study demonstrates the practical application of Python tuples, lists, loops, conditional statements, and arithmetic operators in monitoring food waste.

CASE STUDY 7: STUDENT ACADEMIC RISK CLASSIFICATION

1. Problem Statement :

To develop a Python program that analyses students' marks, attendance percentages, and laboratory scores to classify students into different academic risk categories. The

program should identify students with multiple risk factors and generate a summary of academic performance.

2. Objectives :

1. To store students' marks, attendance, and laboratory scores.

2. To identify students who have low marks, poor attendance, or low laboratory scores.

3. To classify students into low, moderate, and high academic risk categories.

4. To identify students with multiple academic risk factors.

5. To generate a summary of student risk classifications.

6. To understand the use of Python lists, tuples, loops, conditional statements, and logical operators.

3. Assumptions :

1. Student details are stored in a predefined list of tuples.

2. Each tuple contains the student's name, marks, attendance percentage, and laboratory score.

3. Marks and laboratory scores are measured out of 100.

4. Attendance below 75% is considered a risk factor.

5. Marks below 40 are considered a risk factor.

6. Laboratory scores below 40 are considered a risk factor.

7. One risk factor indicates moderate risk, two or more indicate high risk, and no risk factors indicate low risk.

8. These thresholds are assumed for this case study and can be changed according to faculty instructions.

4. Algorithm :

Step 1 : Start.

Step 2 : Initialize the list containing student names, marks, attendance, and laboratory scores.

Step 3 : Set the minimum marks, attendance, and laboratory score thresholds.

Step 4 : Initialize counters for low-risk, moderate-risk, and high-risk students.

Step 5 : Read each student's details.

Step 6 : Check whether marks are below 40, attendance is below 75%, and laboratory score is below 40.

Step 7 : Count the number of risk factors for each student.

Step 8 : If there are two or more risk factors, classify the student as high risk.

Step 9 : If there is exactly one risk factor, classify the student as moderate risk.

Step 10 : If there are no risk factors, classify the student as low risk.

Step 11 : Display the student's risk factors and classification.

Step 12 : Update the category counters and display the final summary.

Step 13 : Stop.

5. Pseudocode :

```
BEGIN
    students = [
        ("Anu", 75, 85, 80),
        ("Ravi", 35, 60, 30),
        ("Sita", 55, 70, 65),
        ("Kiran", 30, 80, 55),
        ("Meena", 65, 90, 35)
    ]
    low_risk = 0
    moderate_risk = 0
    high_risk = 0
    FOR each student IN students
        READ name, marks, attendance, lab_score
        risk_factors = 0
        IF marks < 40 THEN
            risk_factors = risk_factors + 1
        END IF
```

```
        IF attendance < 75 THEN
            risk_factors = risk_factors + 1
        END IF
        IF lab_score < 40 THEN
            risk_factors = risk_factors + 1
        END IF
        IF risk_factors >= 2 THEN
            DISPLAY "High Risk"
            high_risk = high_risk + 1
        ELSE IF risk_factors == 1 THEN
            DISPLAY "Moderate Risk"
            moderate_risk = moderate_risk + 1
        ELSE
            DISPLAY "Low Risk"
            low_risk = low_risk + 1
        END IF
    END FOR
    DISPLAY low_risk
    DISPLAY moderate_risk
    DISPLAY high_risk
END
```

6. Flowchart :

Start

↓

Initialize Student Details and Risk Thresholds

↓

Initialize Risk Category Counters

↓

Read Student Details

↓

Check Marks, Attendance, and Laboratory Score

↓

Count the Number of Risk Factors

↓

Are Risk Factors 2 or More?

- Yes : Classify as High Risk.

- No : Check whether the number of risk factors is 1.

↓

Is the Number of Risk Factors Equal to 1?

- Yes : Classify as Moderate Risk.

- No : Classify as Low Risk.

↓

Update Risk Category Counters

↓

Are All Students Processed?

- No : Read the Next Student.

- Yes : Display the Risk Classification Summary.

↓

Stop

7. Python Source Code :

```
# Student Academic Risk Classification
students = [
    ("Anu", 75, 85, 80),
    ("Ravi", 35, 60, 30),
    ("Sita", 55, 70, 65),
    ("Kiran", 30, 80, 55),
    ("Meena", 65, 90, 35)
]
low_risk = 0
moderate_risk = 0
high_risk = 0
for student in students:
    name, marks, attendance, lab_score = student
    risk_factors = []
    if marks < 40:
        risk_factors.append("Low Marks")
    if attendance < 75:
        risk_factors.append("Low Attendance")
    if lab_score < 40:
        risk_factors.append("Low Laboratory Score")
    print("Student Name:", name)
    print("Marks:", marks)
    print("Attendance:", attendance, "%")
    print("Laboratory Score:", lab_score)
    if len(risk_factors) >= 2:
        print("Risk Factors:", ", ".join(risk_factors))
        print("Risk Category: High Risk")
        high_risk += 1
```

```
    elif len(risk_factors) == 1:
        print("Risk Factors:", ", ".join(risk_factors))
        print("Risk Category: Moderate Risk")
        moderate_risk += 1
    else:
        print("Risk Factors: None")
        print("Risk Category: Low Risk")
        low_risk += 1
print("Academic Risk Summary")
print("Low Risk Students:", low_risk)
print("Moderate Risk Students:", moderate_risk)
print("High Risk Students:", high_risk)
```

8. Test Cases :

Test Case 1 :

Student Name : Anu

Marks : 75

Attendance : 85%

Laboratory Score : 80

Expected Result : Low Risk.

Test Case 2 :

Student Name : Ravi

Marks : 35

Attendance : 60%

Laboratory Score : 30

Expected Result : High Risk because multiple risk factors are present.

Test Case 3 :

Student Name : Sita

Marks : 55

Attendance : 70%

Laboratory Score : 65

Expected Result : Moderate Risk due to low attendance.

Test Case 4 :

Student Name : Kiran

Marks : 30

Attendance : 80%

Laboratory Score : 55

Expected Result : Moderate Risk due to low marks.

Test Case 5 :

Student Name : Meena

Marks : 65

Attendance : 90%

Laboratory Score : 35

Expected Result : Moderate Risk due to a low laboratory score.

9. Output :

```
Student Name: Anu
Marks: 75
Attendance: 85 %
Laboratory Score: 80
Risk Factors: None
Risk Category: Low Risk
Student Name: Ravi
Marks: 35
Attendance: 60 %
Laboratory Score: 30
Risk Factors: Low Marks, Low Attendance, Low Laboratory Score
Risk Category: High Risk
Student Name: Sita
Marks: 55
Attendance: 70 %
Laboratory Score: 65
Risk Factors: Low Attendance
Risk Category: Moderate Risk
Student Name: Kiran
Marks: 30
Attendance: 80 %
Laboratory Score: 55
Risk Factors: Low Marks
Risk Category: Moderate Risk
Student Name: Meena
Marks: 65
Attendance: 90 %
Laboratory Score: 35
Risk Factors: Low Laboratory Score
Risk Category: Moderate Risk
Academic Risk Summary
Low Risk Students: 1
Moderate Risk Students: 3
High Risk Students: 1
```

10. Conclusion :

The Student Academic Risk Classification program successfully analyses students' marks, attendance, and laboratory scores to identify academic risk factors. It classifies students into low, moderate, and high risk categories and displays a summary of the results. This case study demonstrates the practical application of Python tuples, lists, loops, conditional statements, and logical operators in student performance analysis.

CASE STUDY 8: SMART ELECTRICITY BILLING FOR MULTIPLE APARTMENTS

1. Problem Statement :

To develop a Python program that calculates electricity bills for multiple apartments based on their electricity consumption. The program should apply different charges

according to consumption slabs, include a fixed charge, calculate the total bill for each apartment, and display the overall billing summary.

2. Objectives :

1. To store apartment numbers and electricity consumption using Python tuples.

2. To calculate electricity charges based on consumption slabs.

3. To include fixed charges in each apartment's bill.

4. To calculate the total electricity bill for each apartment.

5. To identify apartments with higher electricity consumption.

6. To display the total electricity consumption and overall billing amount.

7. To understand the use of Python tuples, loops, conditional statements, and arithmetic operators.

3. Assumptions :

1. Electricity consumption details for five apartments are stored in a predefined list of tuples.

2. Each tuple contains the apartment number and electricity units consumed.

3. The first 100 units are charged at ₹3 per unit.

4. Units from 101 to 200 are charged at ₹5 per unit.

5. Units above 200 are charged at ₹7 per unit.

6. A fixed charge of ₹50 is added to every apartment's bill.

7. Apartments consuming more than 200 units are classified as high-consumption apartments.

8. All electricity consumption values are assumed to be non-negative.

4. Algorithm :

Step 1 : Start.

Step 2 : Initialize the list containing apartment numbers and electricity consumption.

Step 3 : Set the electricity slab rates and fixed charge.

Step 4 : Initialize the total electricity consumption and total bill to zero.

Step 5 : Read each apartment's details.

Step 6 : Calculate the electricity charge according to the applicable consumption slabs.

Step 7 : Add the fixed charge to calculate the final bill.

Step 8 : Display the apartment number, units consumed, and electricity bill.

Step 9 : If consumption exceeds 200 units, display a high-consumption message.

Step 10 : Update the total electricity consumption and total bill.

Step 11 : Repeat the process for all apartments.

Step 12 : Display the overall electricity consumption and total billing amount.

Step 13 : Stop.

5. Pseudocode :

```
BEGIN
  apartments = [
      ("A101", 120),
      ("A102", 250),
      ("A103", 80),
      ("A104", 310),
      ("A105", 180)
  ]
  total_units = 0
  total_bill = 0
  fixed_charge = 50
  FOR each apartment IN apartments
     READ apartment_number, units
     IF units <= 100 THEN
        charge = units * 3
    ELSE IF units <= 200 THEN
        charge = (100 * 3) + ((units - 100) * 5)
     ELSE
        charge = (100 * 3) + (100 * 5)
              + ((units - 200) * 7)
     END IF
```

```
        bill = charge + fixed_charge
        DISPLAY apartment_number
        DISPLAY units
        DISPLAY bill
        IF units > 200 THEN
            DISPLAY "High Electricity Consumption"
        END IF
        total_units = total_units + units
        total_bill = total_bill + bill
    END FOR
    DISPLAY total_units
    DISPLAY total_bill
END
```

6. Flowchart :

Start

↓

Initialize Apartment Details

↓

Set Electricity Slab Rates and Fixed Charge

↓

Initialize Total Units and Total Bill

↓

Read Apartment Number and Units Consumed

↓

Calculate Electricity Charges According to Slabs

↓

Add Fixed Charge to Calculate Final Bill

↓

Display Apartment Details and Bill

↓

Is Consumption Greater Than 200 Units?

- Yes : Display High Electricity Consumption.

- No : Continue.

↓

Update Total Units and Total Bill

↓

Are All Apartments Processed?

- No : Read the Next Apartment.

- Yes : Display Overall Billing Summary.

↓

Stop

7. Python Source Code :

```
# Smart Electricity Billing for Multiple Apartments
apartments = [
    ("A101", 120),
    ("A102", 250),
    ("A103", 80),
    ("A104", 310),
    ("A105", 180)
]
fixed_charge = 50
total_units = 0
total_bill = 0
for apartment in apartments:
    apartment_number, units = apartment
    if units < 0:
        print("Invalid electricity consumption")
        continue
```

```
    if units <= 100:
        charge = units * 3
    elif units <= 200:
        charge = (100 * 3) + ((units - 100) * 5)
    else:
        charge = (100 * 3) + (100 * 5) + ((units - 200) * 7)
    bill = charge + fixed_charge
    print("Apartment Number:", apartment_number)
    print("Units Consumed:", units)
    print("Electricity Bill: Rs.", bill)
    if units > 200:
        print("High Electricity Consumption")
    total_units += units
    total_bill += bill
print("Overall Billing Summary")
print("Total Units Consumed:", total_units)
print("Total Electricity Bill: Rs.", total_bill)
```

8. Test Cases :

Test Case 1 :

Apartment Number : A101

Units Consumed : 120

Expected Result : Electricity bill is ₹450.

Test Case 2 :

Apartment Number : A102

Units Consumed : 250

Expected Result : Electricity bill is ₹1200 and high electricity consumption is displayed.

Test Case 3 :

Apartment Number : A103

Units Consumed : 80

Expected Result : Electricity bill is ₹290.

Test Case 4 :

Apartment Number : A104

Units Consumed : 310

Expected Result : Electricity bill is ₹1620 and high electricity consumption is displayed.

Test Case 5 :

Apartment Number : A105

Units Consumed : 180

Expected Result : Electricity bill is ₹750.

9. Output :

```
Apartment Number: A101
Units Consumed: 120
Electricity Bill: Rs. 450
Apartment Number: A102
Units Consumed: 250
Electricity Bill: Rs. 1200
High Electricity Consumption
Apartment Number: A103
Units Consumed: 80
Electricity Bill: Rs. 290
Apartment Number: A104
Units Consumed: 310
Electricity Bill: Rs. 1620
High Electricity Consumption
Apartment Number: A105
Units Consumed: 180
Electricity Bill: Rs. 750
Overall Billing Summary
Total Units Consumed: 940
Total Electricity Bill: Rs. 4310
```

10. Conclusion :

The Smart Electricity Billing for Multiple Apartments program successfully calculates electricity bills based on consumption slabs and fixed charges. It identifies apartments with high electricity consumption and displays the total electricity usage and billing amount. This case study demonstrates the practical application of Python tuples, loops, conditional statements, and arithmetic operators in electricity billing management.

CASE STUDY 9: ENVIRONMENTAL MONITORING DATA ANALYZER

1. Problem Statement :

Environmental monitoring helps identify changes in temperature, air quality, and water pH levels. This program analyzes environmental readings from multiple monitoring zones and identifies conditions that need attention.The program calculates average temperature and average Air Quality Index (AQI), detects abnormal readings, and displays a summary of environmental conditions.

2. Objectives :

1. To store environmental readings for multiple monitoring zones.

2. To analyze temperature, air quality, and water pH values.

3. To identify readings that exceed the assumed safe limits.

4. To calculate average temperature and average AQI.

5. To display a summary of zones requiring attention.

3. Assumptions :

1. Data is collected from five monitoring zones.

2. Each zone has temperature, AQI, and water pH readings.

3. Temperature above 40°C is considered high.

4. AQI above 100 is considered poor air quality for this exercise.

5. Water pH below 6.5 or above 8.5 is considered abnormal.

6. These thresholds are simplified assumptions for this assignment and can be changed according to the faculty's instructions.

4. Algorithm :

Step 1 : Start the program.

Step 2 : Store the environmental readings of five monitoring zones in a list of tuples.

Step 3 : Initialize total temperature, total AQI, and the count of zones requiring attention to zero.

Step 4 : Read the temperature, AQI, and pH values of each zone.

Step 5 : Check whether the temperature is above 40°C.

Step 6 : Check whether the AQI is above 100.

Step 7 : Check whether the water pH is below 6.5 or above 8.5.

Step 8 : Display the alerts for each zone.

Step 9 : Calculate the average temperature and average AQI.

Step 10 : Display the overall environmental monitoring summary.

Step 11 : Stop the program.

5. Pseudocode :

```
BEGIN
Create a list of environmental readings
Set total_temperature = 0
Set total_aqi = 0
Set attention_count = 0
FOR each zone in environmental readings
    Extract zone name, temperature, AQI, and pH
    Set alerts to an empty list
    IF temperature > 40
        Add "High Temperature" to alerts
    END IF
    IF AQI > 100
        Add "Poor Air Quality" to alerts
    END IF
    IF pH < 6.5 OR pH > 8.5
        Add "Abnormal Water pH" to alerts
    END IF
    Display zone readings and alerts
    IF alerts is not empty
        Increase attention_count by 1
    END IF
    Add temperature to total_temperature
    Add AQI to total_aqi
```

END FOR

Calculate average temperature

Calculate average AQI

Display averages and attention count

END

6. Flowchart :

Start

↓

Initialize environmental readings and summary variables

↓

Read the environmental data of each zone

↓

Check Temperature > 40°C

↓

Check AQI > 100

↓

Check pH < 6.5 or pH > 8.5

↓

Display environmental alerts for the zone

↓

Update temperature, AQI totals, and attention count

↓

Are all zones processed?

→ No: Process the next zone

→ Yes: Calculate average temperature and average AQI

↓

Display the overall summary

↓

Stop

7. Python Source Code :

```
# Environmental Monitoring Data Analyzer
environment_data = [
    ("Zone A", 32, 85, 7.2),
    ("Zone B", 42, 130, 6.0),
    ("Zone C", 38, 95, 7.5),
    ("Zone D", 41, 110, 8.8),
    ("Zone E", 29, 70, 6.8)
]
total_temperature = 0
total_aqi = 0
attention_count = 0
for zone in environment_data:
    name, temperature, aqi, ph = zone
    alerts = []
    if temperature > 40:
        alerts.append("High Temperature")
    if aqi > 100:
        alerts.append("Poor Air Quality")
    if ph < 6.5 or ph > 8.5:
        alerts.append("Abnormal Water pH")
    print("Monitoring Zone:", name)
    print("Temperature:", temperature, "C")
    print("Air Quality Index:", aqi)
    print("Water pH:", ph)
    if len(alerts) > 0:
        print("Alerts:", ", ".join(alerts))
        attention_count += 1
    else:
```

```
        print("Alerts: None")
    total_temperature += temperature
    total_aqi += aqi
average_temperature = total_temperature / len(environment_data)
average_aqi = total_aqi / len(environment_data)
print("Environmental Monitoring Summary")
print("Average Temperature:", round(average_temperature, 2), "C")
print("Average AQI:", round(average_aqi, 2))
print("Zones Requiring Attention:", attention_count)
```

8. Test Cases :

Test Case 1 :

Normal Environmental Readings

Zone Name : Zone A

Temperature : 32°C

AQI : 85

Water pH : 7.2

Expected Result : No alerts.

Test Case 2 :

All Three Conditions Abnormal

Zone Name : Zone B

Temperature : 42°C

AQI : 130

Water pH : 6.0

Expected Result : High Temperature, Poor Air Quality, and Abnormal Water pH.

Test Case 3 : Normal Readings

Zone Name : Zone C

Temperature : 38°C

AQI : 95

Water pH : 7.5

Expected Result : No alerts.

Test Case 4 :

All Three Conditions Abnormal

Zone Name : Zone D

Temperature : 41°C

AQI : 110

Water pH : 8.8

Expected Result : High Temperature, Poor Air Quality, and Abnormal Water pH.

Test Case 5 :

Normal Environmental Readings

Zone Name : Zone E

Temperature : 29°C

AQI : 70

Water pH : 6.8

Expected Result : No alerts.

9. Output :

```
Monitoring Zone: Zone A
Temperature: 32 C
Air Quality Index: 85
Water pH: 7.2
Alerts: None
Monitoring Zone: Zone B
Temperature: 42 C
Air Quality Index: 130
Water pH: 6.0
Alerts: High Temperature, Poor Air Quality, Abnormal Water pH
Monitoring Zone: Zone C
Temperature: 38 C
Air Quality Index: 95
Water pH: 7.5
Alerts: None
Monitoring Zone: Zone D
Temperature: 41 C
Air Quality Index: 110
Water pH: 8.8
Alerts: High Temperature, Poor Air Quality, Abnormal Water pH
Monitoring Zone: Zone E
Temperature: 29 C
Air Quality Index: 70
Water pH: 6.8
Alerts: None
Environmental Monitoring Summary
Average Temperature: 36.4 C
Average AQI: 98.0
Zones Requiring Attention: 2
```

10. Conclusion :

The Environmental Monitoring Data Analyzer successfully analyzes temperature, air quality, and water pH readings from multiple monitoring zones. It identifies abnormal environmental conditions, displays alerts, and calculates average temperature and AQI values. This program demonstrates the use of Python lists, tuples, loops, conditional statements, operators, and built-in functions for environmental data analysis.

CASE STUDY 10: ADVANCED STRING-BASED REGISTRATION RECORD ANALYZER

1. Problem Statement :

Educational institutions use registration identifiers to maintain student records. Incorrect formats, missing fields, and duplicate registration numbers can create problems in record management.This program analyzes structured student registration strings, validates their format, extracts the admission year and branch code, detects duplicate registration numbers, and classifies invalid records.

2. Objectives :

1. To process student registration records using Python strings.

2. To validate the length and format of registration identifiers.

3. To extract admission year and branch code from valid records.

4. To identify duplicate registration numbers.

5. To classify records as valid, duplicate, or invalid.

3. Assumptions :

1. Each registration record follows the format "YYYY-BR-NNNN".

2. "YYYY" represents the four-digit admission year.

3. "BR" represents a two-letter branch code.

4. "NNNN" represents a four-digit student number.

5. The admission year must contain digits, the branch code must contain uppercase letters, and the student number must contain digits.

6. A registration number appearing more than once is treated as a duplicate.

7. These are assumed rules for this assignment and can be modified according to the institution's actual registration format.

4. Algorithm :

Step 1 : Start the program.

Step 2 : Store student registration records in a list.

Step 3 : Initialize an empty list to store previously processed registration numbers.

Step 4 : Read each registration record.

Step 5 : Split the record using the hyphen separator.

Step 6 : Check whether the record contains exactly three fields.

Step 7 : Validate the admission year, branch code, and student number.

Step 8 : If the format is invalid, classify the record as invalid.

Step 9 : If the format is valid, check whether the registration number already exists in the processed list.

Step 10 : If it exists, classify the record as a duplicate.

Step 11 : Otherwise, classify it as valid, extract its components, and store the registration number.

Step 12 : Display the final record analysis summary.

Step 13 : Stop the program.

5. Pseudocode :

```
BEGIN
Create a list of registration records
Create an empty list called processed_records
Set valid_count = 0
Set duplicate_count = 0
Set invalid_count = 0
FOR each record in registration records
    Split record using "-"
    IF record has exactly three fields THEN
        Extract year, branch, and student number
        Validate:
            Year has four digits
            Branch has two uppercase letters
            Student number has four digits
        IF format is invalid THEN
            Display "Invalid Registration Record"
            Increase invalid_count by 1
        ELSE IF record exists in processed_records THEN
            Display "Duplicate Registration Record"
            Increase duplicate_count by 1
        ELSE
            Add record to processed_records
            Display "Valid Registration Record"
            Display year, branch, and student number
            Increase valid_count by 1
        END IF
```

```
    ELSE
        Display "Invalid Registration Record"
        Increase invalid_count by 1
    END IF
END FOR
Display valid_count
Display duplicate_count
Display invalid_count
END
```

6. Flowchart :

Start

↓

Initialize registration records and counters

↓

Read the next registration record

↓

Split the record into fields

↓

Are there exactly three fields?

→ No: Classify as Invalid

→ Yes: Validate year, branch code, and student number

↓

Is the format valid?

→ No: Classify as Invalid

→ Yes: Check for duplicate registration

↓

Is the registration number already processed?

→ Yes: Classify as Duplicate

→ No: Classify as Valid and extract components

↓

Update counters

↓

Are all records processed?

→ No: Read the next record

→ Yes: Display summary

↓

Stop

7. Python Source Code :

```
# Advanced String-Based Registration Record Analyzer
records = [
    "2025-CS-1001",
    "2025-EC-1002",
    "2025-CS-1001",
    "25-ME-1003",
    "2025-IT-10A4",
    "2024-EE-1005",
    "2025-it-1006"
]
processed_records = []
valid_count = 0
duplicate_count = 0
invalid_count = 0
for record in records:
    print("Registration Record:", record)
    parts = record.split("-")
    if len(parts) != 3:
        print("Status: Invalid Registration Record")
        invalid_count += 1
```

```
        continue
    year, branch, student_number = parts
    if (len(year) != 4 or not year.isdigit()
            or len(branch) != 2
            or not branch.isalpha()
            or not branch.isupper()
            or len(student_number) != 4
            or not student_number.isdigit()):
        print("Status: Invalid Registration Record")
        invalid_count += 1
    elif record in processed_records:
        print("Status: Duplicate Registration Record")
        duplicate_count += 1
    else:
        processed_records.append(record)
        valid_count += 1
        print("Status: Valid Registration Record")
        print("Admission Year:", year)
        print("Branch Code:", branch)
        print("Student Number:", student_number)
print("Registration Analysis Summary")
print("Valid Records:", valid_count)
print("Duplicate Records:", duplicate_count)
print("Invalid Records:", invalid_count)
```

8. Test Cases :

Test Case 1 :

Valid Registration

Registration Record : 2025-CS-1001

Expected Result : Valid registration record.

Test Case 2 :

Duplicate Registration

Registration Record : 2025-CS-1001

Expected Result : Duplicate registration record if the same record was already processed.

Test Case 3 :

Invalid Admission Year

Registration Record : 25-ME-1003

Expected Result : Invalid registration record because the year does not contain four digits.

Test Case 4 :

Invalid Student Number

Registration Record : 2025-IT-10A4

Expected Result : Invalid registration record because the student number contains a letter.

Test Case 5 :

Invalid Branch Code

Registration Record : 2025-it-1006

Expected Result : Invalid registration record because the branch code is not uppercase.

9. Output :

```
Registration Record: 2025-CS-1001
Status: Valid Registration Record
Admission Year: 2025
Branch Code: CS
Student Number: 1001
Registration Record: 2025-EC-1002
Status: Valid Registration Record
Admission Year: 2025
Branch Code: EC
Student Number: 1002
Registration Record: 2025-CS-1001
Status: Duplicate Registration Record
Registration Record: 25-ME-1003
Status: Invalid Registration Record
Registration Record: 2025-IT-10A4
Status: Invalid Registration Record
Registration Record: 2024-EE-1005
Status: Valid Registration Record
Admission Year: 2024
Branch Code: EE
Student Number: 1005
Registration Record: 2025-it-1006
Status: Invalid Registration Record
Registration Analysis Summary
Valid Records: 3
Duplicate Records: 1
Invalid Records: 3
```

10. Conclusion :

The Advanced String-Based Registration Record Analyzer validates student registration identifiers, extracts important components, detects duplicate entries, and classifies invalid records. It demonstrates the use of Python strings, lists, string operations, conditional statements, loops, and counters to maintain accurate registration records.